

Fundamentos para desarrollo de agentes

Fabian Roldan

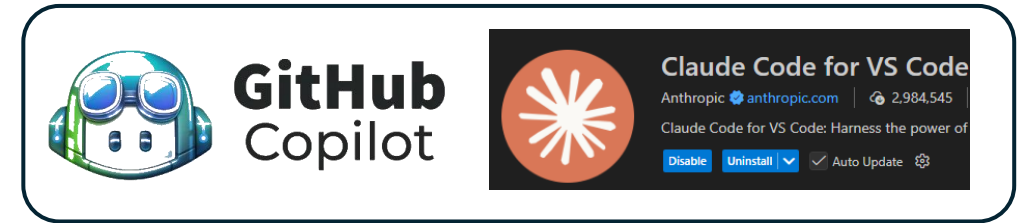
Configuración de entorno de desarrollo

1. De editor de Código (Visual Studio Code) a agentes de código
2. Mapa de ruta a código en producción
3. Sistemas operativos y terminal
4. Gestión de dependencias y entornos virtuales
5. Laboratorios de entornos virtuales y gestión de dependencias
6. Métodos de ejecución de Python

IDEs (Integrated development environment) → **Agentes de código**

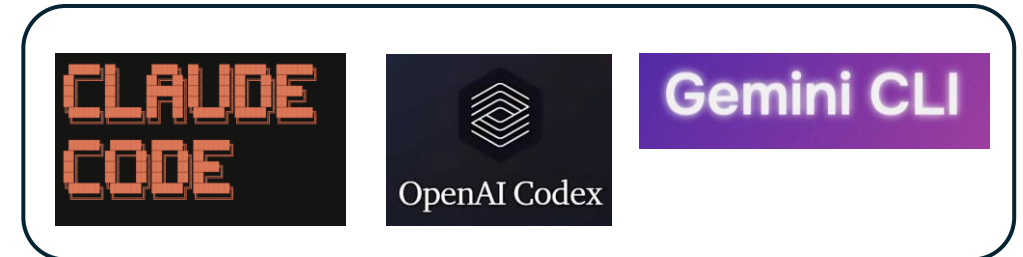
- La Era de los **Plugins / Extensiones** (El Híbrido)

- la IA es un *huésped*
- Software instalado sobre editores clásicos



- **Agentes de Terminal / CLI** (La Revolución "Headless")

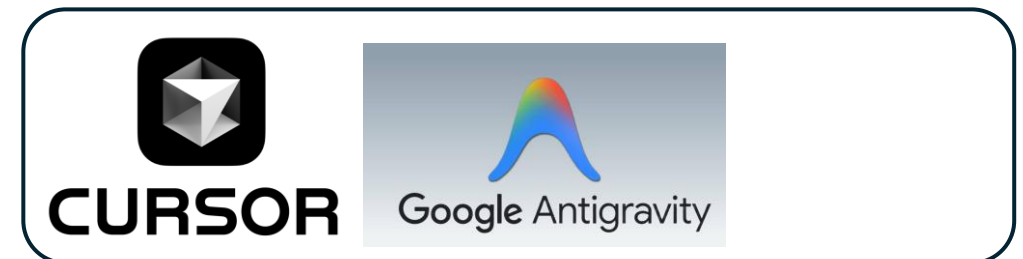
- No usas un editor de texto tradicional
- **Filosofía:** El "editor" es irrelevante. Lo que importa es la manipulación de archivos y la ejecución de comandos



[CLI Agents Explained: How to Choose the Right One](#)

- Espacios de Trabajo Nativos AI y **Agentes Autónomos**

- la IA es el *dueño* de la casa
- Tienen acceso nativo a la terminal, al navegador y a la estructura de archivos.
- Cursor, Windsurf, Gravity (Agent), Devin.
- "Ingeniería de Software Autónoma"



2. Mapa de ruta a código en producción

De notebooks a producción: Hoja de ruta

(Desde la experimentación hasta sistemas en producción)

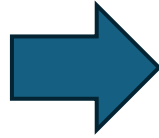
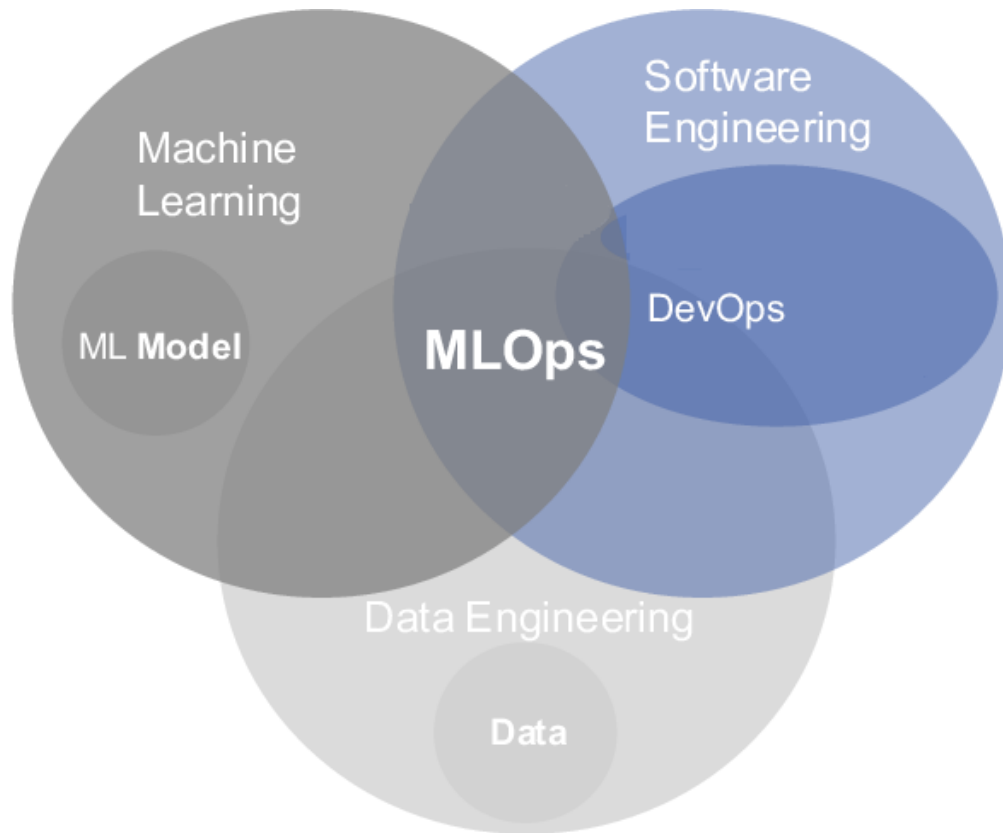
Fase 1: Fundamentos y Calidad del Código



DevOps->MLops->LLMops->AgentOps Fase 2: Automatización y Despliegue



LLMs y Agentes en producción

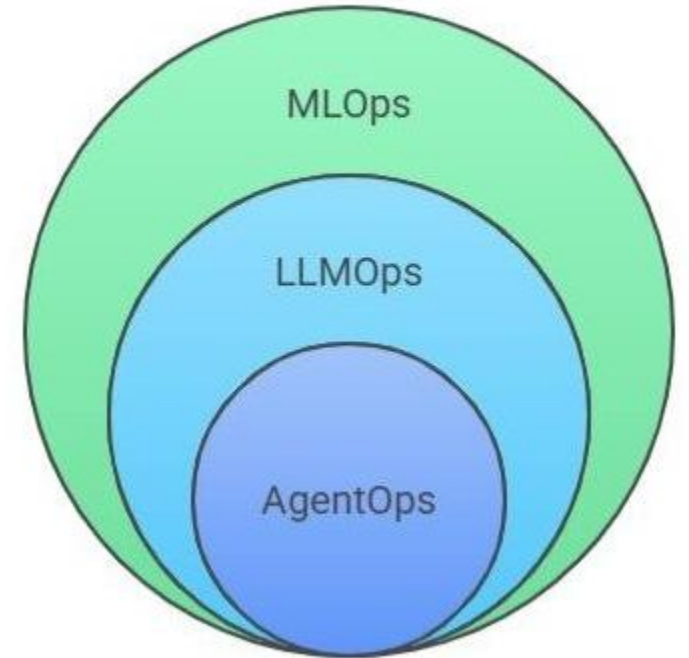


MLOps, LLMOps, and AgentOps Framework

Manages machine learning lifecycle

Optimizes large language model workflows

Streamlines and automates agent-based tasks



De notebooks a producción: Hoja de ruta

(Desde la experimentación hasta sistemas en producción)

Fase 1: Fundamentos y Calidad del Código



DevOps->MLops->LLMops->AgentOps

Fase 2: Automatización y Despliegue



1

Fase de Exploración: Jupyter Notebooks

El punto de partida donde experimentas, creas prototipos y validas tus ideas rápidamente. Los notebooks son perfectos para exploración de datos, visualización e iteración rápida.

▶ ▶ Lo Que Haces

- Análisis exploratorio de datos (EDA)
- Prototipado rápido
- Visualizaciones y gráficos
- Experimentación de modelos
- Depuración interactiva

▶ ▶ Características

- Flujo de ejecución lineal
- Salidas y gráficos en línea
- Documentación en Markdown 
- Ciclos de retroalimentación rápidos
- Entorno de usuario único

Jupyter Notebook

JupyterLab

Google Colab

Kaggle Notebooks

✓ Perfecto Para:

Investigación, aprendizaje, exploración de datos, pruebas de concepto, compartir resultados con stakeholders



Organización del Código: Refactorización a Archivos .py

Transforma el código del notebook en módulos Python estructurados y reutilizables. Aquí empiezas a pensar en calidad del código, mantenibilidad y modularidad.

▶ ▶ Acciones Clave

- Extraer funciones de las celdas
- Crear módulos Python (archivos .py)
- Separar lógica de procesamiento de datos
- Organizar en carpetas/paquetes
- Eliminar visualizaciones en línea

▶ ▶ Ejemplo de Estructura

- src/data_processing.py
- src/model.py
- src/utils.py
- src/config.py
- tests/

[VSCode](#)[PyCharm](#)[nbconvert](#)[jupyter](#)

⚠ Desafíos Comunes:

Romper dependencias entre celdas, manejar estado global, eliminar elementos interactivos

Estructura del Proyecto: Configuración de Arquitectura Adecuada

Construye una estructura de proyecto profesional con separación adecuada de responsabilidades, gestión de configuración y manejo de dependencias.

► ► Diseño del Proyecto

- requirements.txt / pyproject.toml
- README.md con documentación 
- .gitignore para control de versiones
- Archivos de configuración (YAML/JSON) 
- Variables de entorno (.env)

► ► Mejores Prácticas

- Separar datos del código
- Usar entornos virtuales
- Agregar logging en lugar de prints
- Gestión de configuración
- Manejo de errores y validación

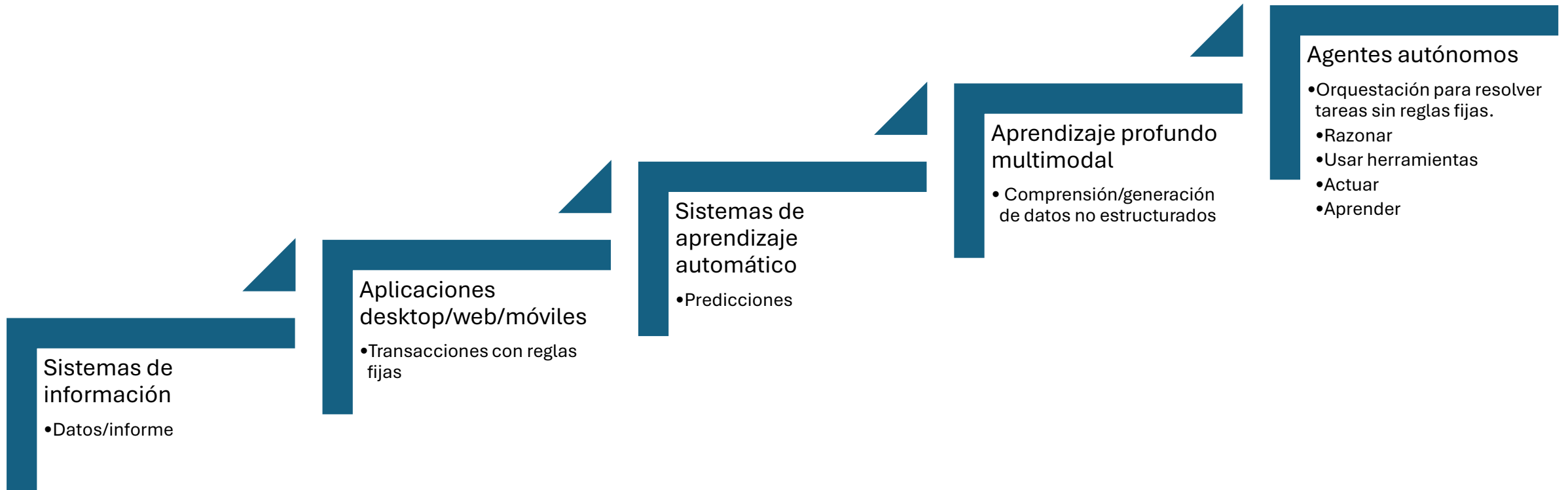
Poetry

pip-tools

cookiecutter

pydantic

Cambio de paradigma en las aplicaciones



3. Sistemas operativos y terminal

- Jeff Bezos insiste en que **entender lo que permanece constante en los negocios es crucial para el éxito.**

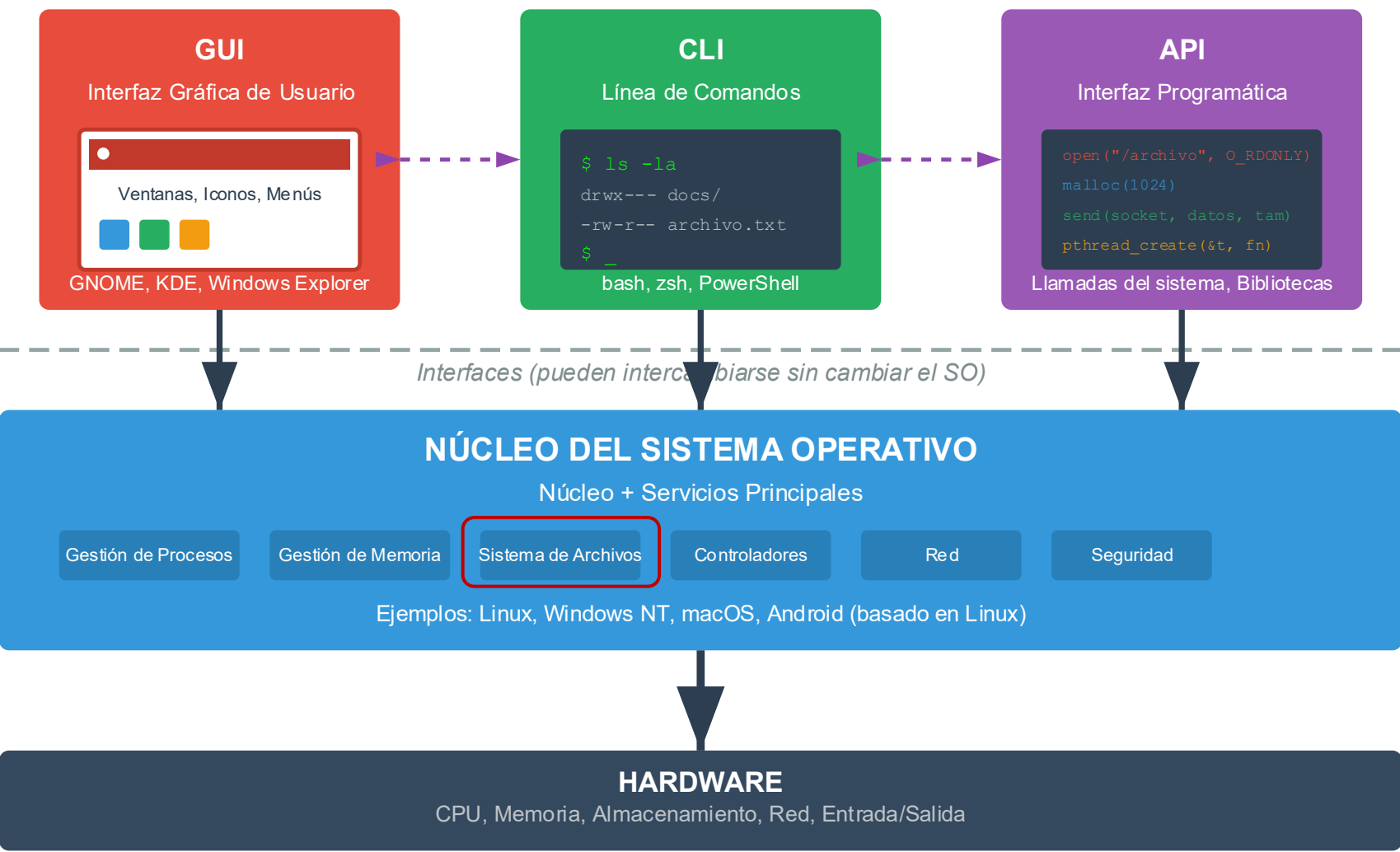
“Cree que centrarse en elementos que los clientes siempre valorarán -como los **precios bajos**, la **entrega rápida** y una **amplia selección**- ayuda a las empresas a hacer mejores planes e inversiones....”



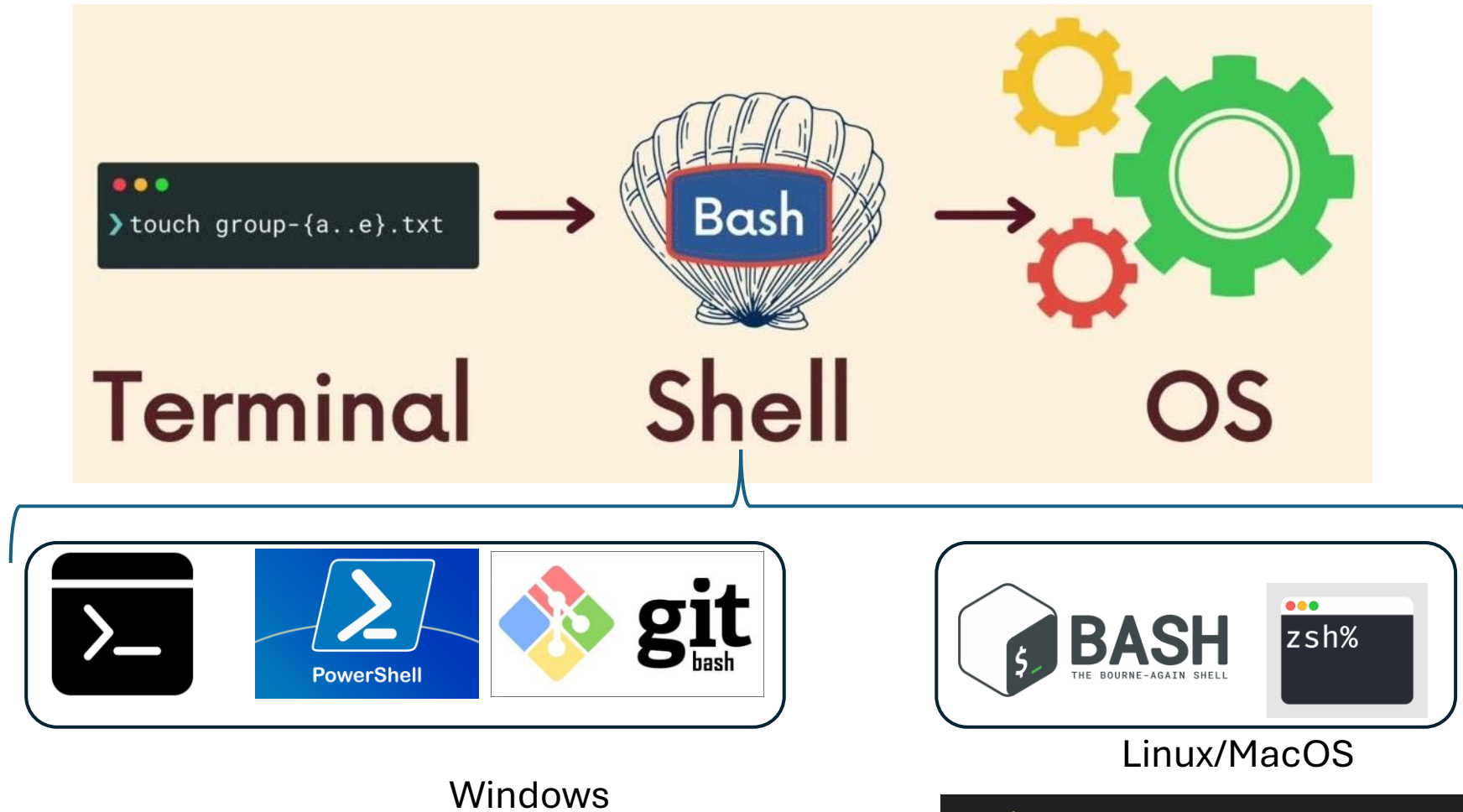
CLI (command line interface)

Sistema Operativo: Núcleo + Interfaces

El mismo núcleo de Linux puede ejecutarse con GUI GNOME o solo CLI



SHELLs para la terminal de comandos



```
echo $0
echo $SHELL
ps -p $$
chsh -s /bin/bash    # Para cambiar a bash
chsh -s /bin/zsh     # Para cambiar a zsh
```

Anatomía de un comando en CLI

```
comando [opciones] [argumentos]
```

1 El Comando (El Verbo - La Acción)

Es la acción que quieres ejecutar. Es **siempre** lo primero que escribes.

2 Las Opciones/Flags (Los Modificadores)

Modifican **cómo** se ejecuta el comando. Empiezan con `-` (corto) o `--` (largo).

Reglas universales:

- `-x` → Opción corta (una letra)
- `--palabra` → Opción larga (palabra completa)
- `-xyz` → Combinar múltiples opciones cortas (equivale a `-x -y -z`)
- `--option=value` → Opción con valor asignado

3 Los Argumentos (Los Objetos)

Son **sobre qué** actúa el comando. Pueden ser archivos, carpetas, texto, etc.

Categorías de comandos

Categoría	Comandos Clave
Navegación	<code>pwd</code> , <code>ls</code> , <code>cd</code>
Archivos	<code>mkdir</code> , <code>touch</code> , <code>cp</code> , <code>mv</code> , <code>rm</code>
Visualización	<code>cat</code> , <code>head</code> , <code>tail</code> , <code>less</code>
Búsqueda	<code>grep</code> , <code>find</code>
Variables	<code>echo \$VAR</code> , <code>export</code> , <code>\$env:VAR</code>
Procesos	<code>ps</code> , <code>top</code> , <code>htop</code> , <code>kill</code>
Git	<code>init</code> , <code>add</code> , <code>commit</code> , <code>push</code> , <code>pull</code>
Red	<code>curl</code> , <code>wget</code> , <code>ping</code> , <code>netstat</code>

Navegación y gestión de archivos

Navegación Básica

Tarea	Linux/macOS	Git Bash	PowerShell	Explicación
Ver directorio actual	<code>pwd</code>	<code>pwd</code>	<code>pwd</code> o <code>Get-Location</code>	Print Working Directory
Listar archivos	<code>ls</code>	<code>ls</code>	<code>ls</code> o <code>dir</code>	Lista archivos y carpetas
Lista detallada	<code>ls -l</code>	<code>ls -l</code>	<code>ls</code>	Muestra permisos, tamaño, fecha
Incluir ocultos	<code>ls -a</code>	<code>ls -a</code>	<code>ls -Force</code>	Muestra archivos que empiezan con .
Lista completa	<code>ls -la</code>	<code>ls -la</code>	<code>ls -Force</code>	Detallado + ocultos
Lista recursiva	<code>ls -R</code>	<code>ls -R</code>	<code>ls -Recurse</code>	Muestra subdirectorios también
Cambiar directorio	<code>cd carpeta</code>	<code>cd carpeta</code>	<code>cd carpeta</code>	Change Directory
Subir un nivel	<code>cd ..</code>	<code>cd ..</code>	<code>cd ..</code>	Al directorio padre
Ir a home	<code>cd ~</code>	<code>cd ~</code>	<code>cd \$HOME</code>	Tu directorio de usuario
Ir a raíz	<code>cd /</code>	<code>cd /</code>	<code>cd \</code>	Raíz del sistema
Volver a anterior	<code>cd -</code>	<code>cd -</code>	<code>cd -</code>	Directorio previo

Navegación y gestión de archivos

Gestión de Directorios

Tarea	Linux/macOS	Git Bash	PowerShell	Explicación
Crear directorio	<code>mkdir carpeta</code>	<code>mkdir carpeta</code>	<code>mkdir carpeta</code>	Make Directory
Crear anidados	<code>mkdir -p ruta/a/carpeta</code>	<code>mkdir -p ruta/a/carpeta</code>	<code>mkdir -Force ruta\carpeta</code>	Crea padres si no existen
Eliminar vacío	<code>rmdir carpeta</code>	<code>rmdir carpeta</code>	<code>rmdir carpeta</code>	Solo si está vacía
Eliminar con contenido	<code>rm -r carpeta</code>	<code>rm -r carpeta</code>	<code>rm -r carpeta</code>	Recursivo (todo dentro)
Forzar eliminación	<code>rm -rf carpeta</code>	<code>rm -rf carpeta</code>	<code>rm -Force -Recurse carpeta</code>	Sin confirmación
Copiar directorio	<code>cp -r origen destino</code>	<code>cp -r origen destino</code>	<code>cp -r origen destino</code>	Copia todo el contenido
Mover/renombrar	<code>mv viejo nuevo</code>	<code>mv viejo nuevo</code>	<code>mv viejo nuevo</code>	Move (también renombra)

Navegación y gestión de archivos



Gestión de Archivos

Tarea	Linux/macOS	Git Bash	PowerShell	Explicación
Crear archivo vacío	<code>touch archivo.py</code>	<code>touch archivo.py</code>	<code>ni archivo.py</code> o <code>New-Item</code>	New Item
Copiar archivo	<code>cp origen.py destino.py</code>	<code>cp origen.py destino.py</code>	<code>cp origen.py destino.py</code>	Copy
Copiar múltiples	<code>cp *.py backup/</code>	<code>cp *.py backup/</code>	<code>cp *.py backup/</code>	Usa wildcards
Mover archivo	<code>mv viejo.py nuevo.py</code>	<code>mv viejo.py nuevo.py</code>	<code>mv viejo.py nuevo.py</code>	Move (renombrar)
Eliminar archivo	<code>rm archivo.py</code>	<code>rm archivo.py</code>	<code>rm archivo.py</code>	Remove
Eliminar múltiples	<code>rm *.log</code>	<code>rm *.log</code>	<code>rm *.log</code>	Elimina todos los .log
Forzar eliminación	<code>rm -f archivo</code>	<code>rm -f archivo</code>	<code>rm -Force archivo</code>	Sin confirmación

Linux en Windows



1. Instalar WSL en Window

```
#Windows Subsystem for Linux

#launch powershell as administrator
wsl --install #engine
wsl --list --online
wsl --list --verbose
wsl --install -d Ubuntu #engine + distribution

wsl --version
wsl --status

#To run/start wsl/ubuntu: Open powershell
wsl
#username and password
#home directory
cd ~
pwd
ls

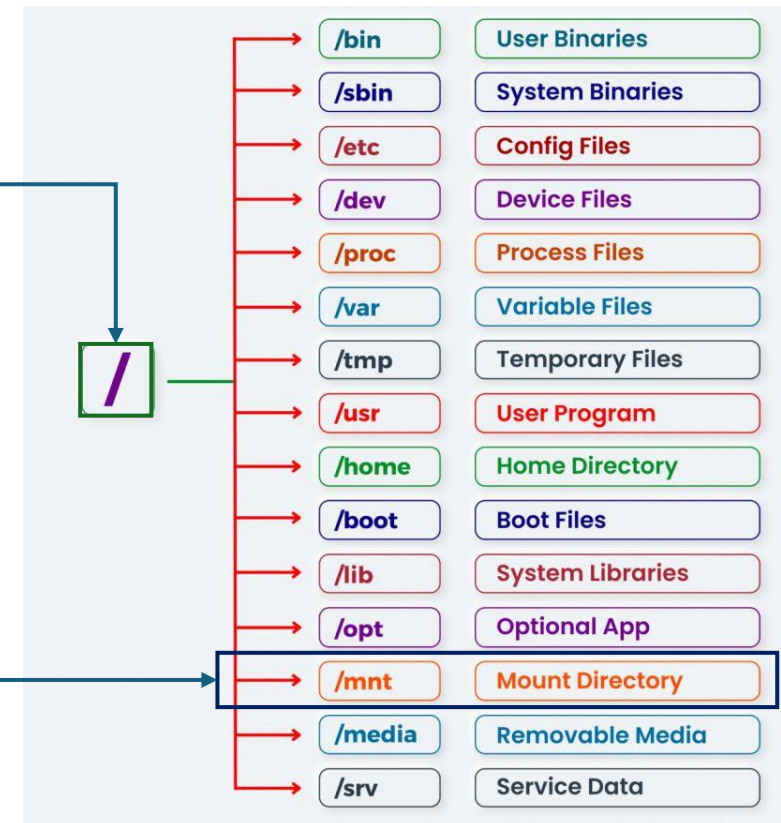
#system directories
cd ..
cd ls

#To stop wsl
exit
```

```
PS C:\Users\xfrol> wsl
xfrol@FABIANSURFACE:/mnt/c/Users/xfrol$ cd
xfrol@FABIANSURFACE:~$ pwd
/home/xfrol
xfrol@FABIANSURFACE:~$ cd /
xfrol@FABIANSURFACE:/$ pwd
/
xfrol@FABIANSURFACE:/$ ls
bin          init          opt           snap
bin.usr-is-merged  lib           proc          srv
boot         lib.usr-is-merged  root          sys
dev          lost+found      run           tmp
etc          media           sbin          usr
home        mnt             sbin.usr-is-merged  var
xfrol@FABIANSURFACE:/$
```

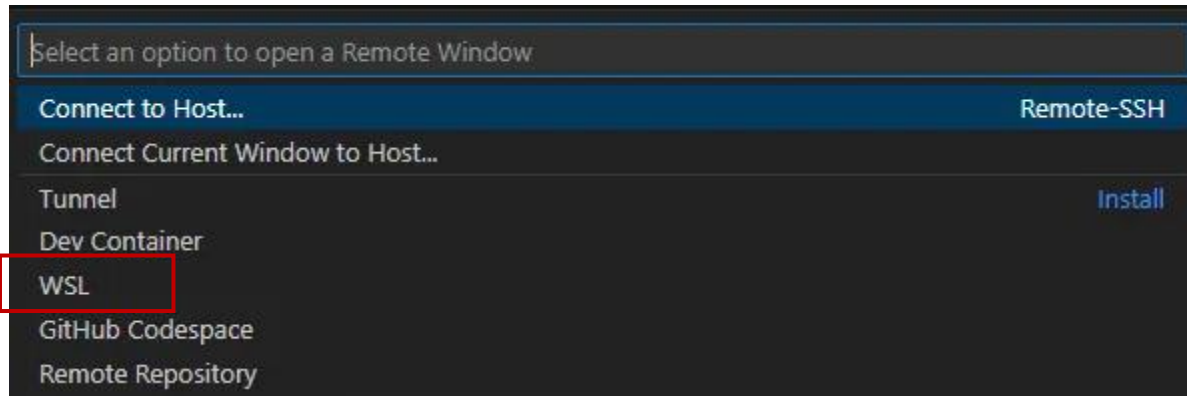
```
PS C:\Windows\system32> wsl
xfrol@FABIANSURFACE:/mnt/c/Windows/system32$
```

Veamos el sistema de archivos de Linux

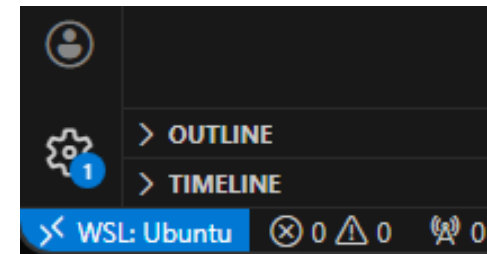


Linux en Windows

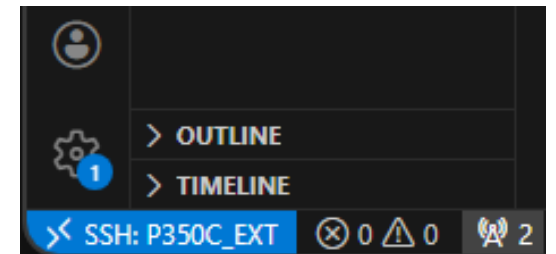
2. Crear conexión de Visual Studio Code a WSL.



Desarrollo local
en Linux



Desarrollo
remoto en Linux



3. Crear directorio para proyectos.

```
#create directory for repositories and projects
pwd
mkdir repos
cd repos
```

4. Gestión de dependencias y entornos virtuales

Gestores de paquetes



Stack Tecnológico Necesario

- **LangChain:** Framework de agentes
- **OpenAI/Anthropic:** APIs de LLM
- **ChromaDB/Pinecone:** Bases vectoriales
- **Pandas/NumPy:** Análisis de datos
- **FastAPI:** Servidor web
- **Pydantic:** Validación de datos

~50+ paquetes Python



El Desafío Real

- ✗ Versiones incompatibles entre proyectos
- ✗ Conflictos de dependencias
- ✗ "Funciona en mi máquina" pero no en producción
- ✗ Sistema Python contaminado
- ✗ Imposible reproducir entornos
- ✗ Colaboración en equipo compleja



La Solución: Package Management Profesional



Paquetes

Código reutilizable



Gestores

5 tipos diferentes



Entornos

Aislamiento total

Gestores de paquetes



Arquitectura en Capas

Cómo se integran los diferentes gestores

CAPA 1: Sistema Base

SYS-PM Instala Git, Python, Build Tools → apt, brew, winget



CAPA 2: Múltiples Versiones Python

VER-MGR Gestiona Python 3.9, 3.10, 3.11, 3.12 → pyenv



CAPA 3: Entornos Aislados

ENV-MGR Crea .venv/ por proyecto → venv



CAPA 4: Paquetes Python

LANG-PM Instala langchain, fastapi, etc. → pip

Gestores de paquetes

Arquitectura en Capas

Cómo se integran los diferentes gestores



System PM SYS-PM

Qué hace: Gestiona software a nivel del sistema operativo

Ejemplos:

- apt (Ubuntu/Debian)
- brew (macOS)
- winget (Windows)

```
sudo apt install python3
brew install git
```

Paquete	Qué incluye	Para qué sirve
tree	tree	Ver estructura de carpetas
htop	htop	Monitor de procesos mejorado
rsync	rsync	Sincronizar archivos

SQLite (Base de datos local, simple)

Paquete	Qué incluye	Para qué sirve
sqlite3	sqlite3	Base de datos embebida en archivo

PostgreSQL (Base de datos profesional)

Paquete	Qué incluye	Para qué sirve
postgresql	servidor PostgreSQL	Base de datos relacional potente
postgresql-client	psql	Cliente para conectarse
libpq-dev	headers	Para instalar psycopg2 en Python

Gestores de paquetes

Arquitectura en Capas

Cómo se integran los diferentes gestores



Comandos APT (Gestor de Paquetes)

APT es el comando principal para instalar, actualizar y eliminar programas en Ubuntu/Debian.

Comando	Para qué sirve
<code>sudo apt update</code>	Actualizar lista de paquetes disponibles
<code>sudo apt upgrade</code>	Actualizar todos los paquetes instalados
<code>sudo apt install <paquete></code>	Instalar un paquete
<code>sudo apt install -y <paquete></code>	Instalar sin pedir confirmación
<code>sudo apt remove <paquete></code>	Desinstalar un paquete
<code>sudo apt purge <paquete></code>	Desinstalar + eliminar configuración
<code>sudo apt autoremove</code>	Eliminar paquetes que ya no se necesitan
<code>sudo apt search <término></code>	Buscar paquetes
<code>apt show <paquete></code>	Ver información de un paquete
<code>apt list --installed</code>	Ver todos los paquetes instalados
<code>apt list --installed grep <nombre></code>	Buscar un paquete instalado

Gestores de paquetes



Arquitectura en Capas

Cómo se integran los diferentes gestores

CAPA 1: Sistema Base

SYS-PM Instala Git, Python, Build Tools → apt, brew, winget



CAPA 2: Múltiples Versiones Python

VER-MGR Gestiona Python 3.9, 3.10, 3.11, 3.12 → pyenv



CAPA 3: Entornos Aislados

ENV-MGR Crea .venv/ por proyecto → venv



CAPA 4: Paquetes Python

LANG-PM Instala langchain, fastapi, etc. → pip



Version Manager

VER-MGR

Qué hace: Gestiona múltiples versiones del lenguaje

Ejemplos:

- pyenv (Python)
- nvm (Node.js)
- rustup (Rust)

```
pyenv install 3.11.6  
pyenv global 3.11.6
```

Gestores de paquetes



Arquitectura en Capas

Cómo se integran los diferentes gestores

CAPA 1: Sistema Base

SYS-PM Instala Git, Python, Build Tools → apt, brew, winget



CAPA 2: Múltiples Versiones Python

VER-MGR Gestiona Python 3.9, 3.10, 3.11, 3.12 → pyenv



CAPA 3: Entornos Aislados

ENV-MGR Crea .venv/ por proyecto → venv



CAPA 4: Paquetes Python

LANG-PM Instala langchain, fastapi, etc. → pip



Environment Manager **ENV-MGR**

Qué hace: Crea entornos virtuales aislados

Ejemplos:

- venv (Python built-in)
- virtualenv (Python)
- poetry (híbrido)

```
python -m venv .venv
source .venv/bin/activate
```

Gestores de paquetes



Arquitectura en Capas

Cómo se integran los diferentes gestores

CAPA 1: Sistema Base

SYS-PM Instala Git, Python, Build Tools → apt, brew, winget



CAPA 2: Múltiples Versiones Python

VER-MGR Gestiona Python 3.9, 3.10, 3.11, 3.12 → pyenv



CAPA 3: Entornos Aislados

ENV-MGR Crea .venv/ por proyecto → venv



CAPA 4: Paquetes Python

LANG-PM Instala langchain, fastapi, etc. → pip



Language PM

LANG-PM

Qué hace: Instala paquetes específicos del lenguaje

Ejemplos:

- pip (Python)
- npm (JavaScript)
- cargo (Rust)

Gestores de paquetes

```
pip install langchain
npm install react
```

Gestores de paquetes



Arquitectura en Capas

Cómo se integran los diferentes gestores

CAPA 1: Sistema Base

SYS-PM Instala Git, Python, Build Tools → apt, brew, winget



CAPA 2: Múltiples Versiones Python

VER-MGR Gestiona Python 3.9, 3.10, 3.11, 3.12 → pyenv



CAPA 3: Entornos Aislados

ENV-MGR Crea .venv/ por proyecto → venv



CAPA 4: Paquetes Python

LANG-PM Instala langchain, fastapi, etc. → pip

python3 / python3-dev

Intérprete Python y headers para compilar extensiones nativas

```
sudo apt install python3 python3-dev
```

python3-pip

Gestor de paquetes Python (si no viene incluido)

```
sudo apt install python3-pip
```

python3-venv

Módulo para crear entornos virtuales (Debian/Ubuntu)

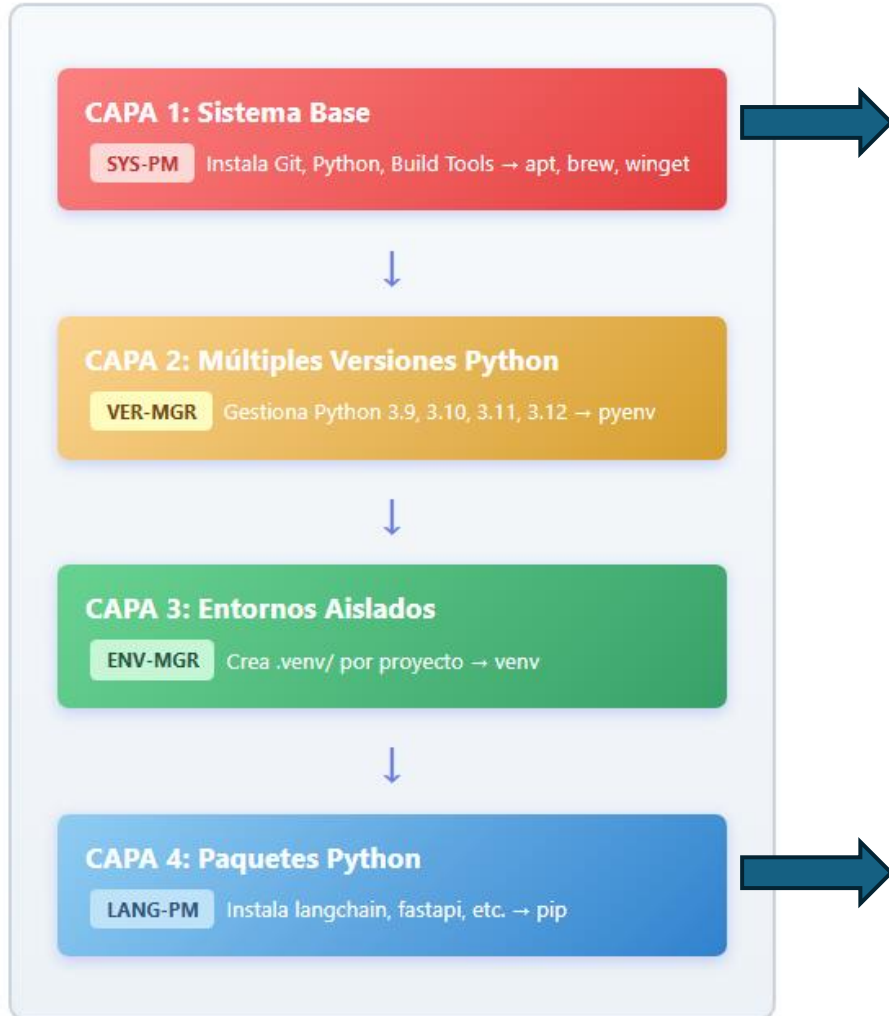
```
sudo apt install python3-venv
```


Gestores de paquetes



Arquitectura en Capas

Cómo se integran los diferentes gestores



Paquete de Sistema Linux (apt/dpkg):

- Son programas completos (git, nginx, postgresql)
- Se instalan con `apt install`
- Van a `/usr/bin`, `/usr/lib`, etc.

Paquete/Módulo de Python (pip):

- Son librerías de Python (langchain, openai, pandas)
- Se instalan con `pip install`
- Van a `/usr/lib/python3.12/` o `.venv/lib/python3.12/`

Gestores de paquetes



Arquitectura en Capas

Cómo se integran los diferentes gestores

CAPA 1: Sistema Base

SYS-PM Instala Git, Python, Build Tools → apt, brew, winget



CAPA 2: Múltiples Versiones Python

VER-MGR Gestiona Python 3.9, 3.10, 3.11, 3.12 → pyenv



CAPA 3: Entornos Aislados

ENV-MGR Crea .venv/ por proyecto → venv



CAPA 4: Paquetes Python

LANG-PM Instala langchain, fastapi, etc. → pip



Comando

Para qué sirve

```
pip3 list
```

Ver paquetes Python instalados

```
pip3 install <paquete>
```

Instalar un paquete

```
pip3 install -r requirements.txt
```

Instalar desde archivo

```
pip3 uninstall <paquete>
```

Desinstalar un paquete

```
pip3 show <paquete>
```

Ver información de un paquete

```
pip3 freeze
```

Listar paquetes con versiones

```
pip3 freeze > requirements.txt
```

Guardar lista de paquetes

```
pip3 search <término>
```

Buscar paquetes (deprecado)

```
python3 -m pip install <paquete>
```

MEJOR PRÁCTICA

Gestores de paquetes



Arquitectura en Capas

Cómo se integran los diferentes gestores

CAPA 1: Sistema Base

SYS-PM Instala Git, Python, Build Tools → apt, brew, winget



CAPA 2: Múltiples Versiones Python

VER-MGR Gestiona Python 3.9, 3.10, 3.11, 3.12 → pyenv



CAPA 3: Entornos Aislados

ENV-MGR Crea .venv/ por proyecto → venv



CAPA 4: Paquetes Python

LANG-PM Instala langchain, fastapi, etc. → pip



Hybrid PM

HYBRID

Qué hace: Combina múltiples funcionalidades (versiones + entornos + paquetes)

Ejemplos:

- **conda:** Todo-en-uno para Data Science (Python + R + librerías C++)
- **poetry:** Gestión moderna para Python (lock files + publicación)

```
# Conda
conda create -n myenv python=3.11
conda install pandas numpy

# Poetry
poetry new myproject
poetry add langchain
```

Gestión de dependencias en Python (Gestores híbridos)



[Poetry vs Pip: Choosing the Right Python Package Manager | Better Stack Community](#)

[Conda vs Pip: Choosing the Right Python Package Manager | Better Stack Community](#) [Conda_Cheat_Sheet_1.pdf](#)

[Python Poetry: Modern And Efficient Python Environment And Dependency Management | DataCamp](#)

Gestión de dependencias en Python (Gestores híbridos)



VS



¿Cuándo elegir Conda?

- **Proyectos multilingües:** sistemas que utilizan componentes Python, R y C++.
- **Cargas de trabajo aceleradas por GPU:** proyectos que requieren paquetes CUDA optimizados.
- **Computación científica:** proyectos de investigación que utilizan paquetes científicos especializados.
- **Principiantes:** configuración inicial más sencilla con la distribución Anaconda.

¿Cuándo elegir Poetry?

- **Proyectos de ML en Python puro:** sistemas de IA que solo utilizan Python.
- **Pipelines de CI/CD:** compilaciones e implementaciones más rápidas.
- **Desarrollo de paquetes:** equipos que crean y publican paquetes de ML.
- **Flujos de trabajo modernos:** proyectos que utilizan flujos de trabajo basados en Git e implementaciones de contenedores.

Gestión de dependencias en Python (Gestores híbridos)

The Conda logo, featuring the word "CONDA" in green capital letters. The letter "C" is stylized with a green circular icon containing a white grid pattern.

VS

The Poetry logo, featuring a blue and purple abstract icon to the left of the word "Poetry" in blue lowercase letters.

Característica	Conda	Poetry
Scope	Científico (data science)	Desarrollo Python general
Gestiona	Python + binarios + R	Solo Python
Repositorio	Anaconda + PyPI	Solo PyPI
Archivo config	<code>environment.yml</code>	<code>pyproject.toml</code>
Lock file	No nativo	<code>poetry.lock</code>
Velocidad	Lento (resuelve todo)	Rápido
Tamaño	Grande (Anaconda ~3GB)	Ligero



Windows Command Prompt Windows PowerShell macOS **Linux**

1. Run the following four commands to download and install the latest Linux installer for your chosen chip architecture. Line by line, these commands:

- create a new directory named "miniconda3" in your home directory.
- download the Linux Miniconda installation script for your chosen chip architecture and save the script as `miniconda.sh` in the miniconda3 directory.
- run the `miniconda.sh` installation script in silent mode using `bash`.
- remove the `miniconda.sh` installation script file after installation is complete.

64-bit **AWS Graviton 2/ARM 64** IBM Z

```
mkdir -p ~/miniconda3
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3
rm ~/miniconda3/miniconda.sh
```

```
xfrol@FABIANSURFACE:~$ pwd
/home/xfrol
xfrol@FABIANSURFACE:~$ mkdir -p ~/miniconda3
xfrol@FABIANSURFACE:~$ ls
miniconda3  repos
xfrol@FABIANSURFACE:~$ wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-aarch64.sh -O ~/miniconda3/miniconda.sh
--2026-01-16 03:55:01-- https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-aarch64.sh
Resolving repo.anaconda.com (repo.anaconda.com)... 104.16.191.158, 104.16.32.241, 2606:4700::6810:20f1, ...
Connecting to repo.anaconda.com (repo.anaconda.com)|104.16.191.158|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 156445478 (149M) [application/octet-stream]
Saving to: '/home/xfrol/miniconda3/miniconda.sh'

/home/xfrol/miniconda3/miniconda.sh      100%[=====] 149.20M  38.7MB/s   in 5.2s

2026-01-16 03:55:07 (28.9 MB/s) - '/home/xfrol/miniconda3/miniconda.sh' saved [156445478/156445478]

xfrol@FABIANSURFACE:~$ bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3
PREFIX=/home/xfrol/miniconda3
Unpacking bootstrapper...
Unpacking payload...

Installing base environment...

Preparing transaction: ...working... done
Executing transaction: ...working... done
installation finished.
xfrol@FABIANSURFACE:~$ rm ~/miniconda3/miniconda.sh
xfrol@FABIANSURFACE:~$ conda --version
conda: command not found
xfrol@FABIANSURFACE:~$ source ~/miniconda3/bin/activate
(base) xfrol@FABIANSURFACE:~$ conda --version
conda 25.11.1
```



Installation



If you are viewing documentation for the development branch, you may wish to install a preview or development version of Poetry. See the **advanced** installation instructions to use a preview or alternate version of Poetry.

With pipx

With the official installer

Manually (advanced)

CI recommendations

We provide a custom installer that will install Poetry in a new virtual environment and allows Poetry to manage its own environment.

1 Install Poetry

The installer script is available directly at install.python-poetry.org, and is developed in its own repository. The script can be executed directly (i.e. 'curl python') or downloaded and then executed from disk (e.g. in a CI environment).

Linux, macOS, Windows (WSL)

```
curl -sSL https://install.python-poetry.org | python3 -
```



```
• (base) xfrol@FABIANSURFACE:~$ curl -sSL https://install.python-poetry.org | python3 -  
Retrieving Poetry metadata
```

```
# Welcome to Poetry!
```

```
This will download and install the latest version of Poetry,  
a dependency and package manager for Python.
```

```
It will add the `poetry` command to Poetry's bin directory, located at:
```

```
/home/xfrol/.local/bin
```

```
You can uninstall at any time by executing this script with the --uninstall option,  
and these changes will be reverted.
```

```
Installing Poetry (2.2.1): Done
```

```
Poetry (2.2.1) is installed now. Great!
```

```
To get started you need Poetry's bin directory (/home/xfrol/.local/bin) in your `PATH`  
environment variable.
```

```
Add `export PATH="/home/xfrol/.local/bin:$PATH"` to your shell configuration file.
```

```
Alternatively, you can call Poetry explicitly with /home/xfrol/.local/bin/poetry.
```

```
You can test that everything is set up by executing:
```

```
`poetry --version`
```

[Introduction](#) | [Documentation](#) | [Poetry - Python dependency management and packaging made easy](#)

Gestores de paquetes



Arquitectura en Capas

Cómo se integran los diferentes gestores

CAPA 1: Sistema Base

SYS-PM Instala Git, Python, Build Tools → apt, brew, winget



CAPA 2: Múltiples Versiones Python

VER-MGR Gestiona Python 3.9, 3.10, 3.11, 3.12 → pyenv



CAPA 3: Entornos Aislados

ENV-MGR Crea .venv/ por proyecto → venv



CAPA 4: Paquetes Python

LANG-PM Instala langchain, fastapi, etc. → pip

1. System PM instala Python intérprete
 - └ `sudo apt install python3.11` (Linux)
 - └ `brew install python@3.11` (macOS)
 - └ `winget install Python.Python.3.11` (Windows)
2. (Opcional) Version Manager gestiona múltiples Pythons
 - └ `pyenv install 3.11.0`
 - └ `pyenv global 3.11.0`
3. Crear entorno virtual (SIEMPRE)
 - └ `python -m venv .venv`
 - └ `source .venv/bin/activate`
4. Language PM instala librerías EN EL ENTORNO
 - └ `pip install langchain`
 - └ `pip install -r requirements.txt`
5. (Alternativa) Usar Hybrid PM
 - └ `conda create -n myenv python=3.11`
 - └ `conda install numpy pandas`
 - └ `pip install langchain`

Gestión de dependencias en Python (Gestores híbridos)

The logo for Conda, featuring the word "CONDA" in a bold, green, sans-serif font. The letter "C" is stylized with a circular pattern of small triangles.

```
# Crear entorno
conda create -n myenv python=3.11

# Activar entorno
conda activate myenv

# Instalar paquetes
conda install numpy pandas scikit-learn

# Instalar desde PyPI (si no está en conda)
pip install langchain

# Listar entornos
conda env list

# Listar paquetes en entorno actual
conda list

# Exportar entorno
conda env export > environment.yml

# Crear desde archivo
conda env create -f environment.yml

# Desactivar
conda deactivate

# Eliminar entorno
conda env remove -n myenv
```



```
# Inicializar proyecto nuevo
poetry new mi-proyecto

# O agregar a proyecto existente
poetry init

# Instalar dependencias
poetry add langchain

# Instalar dev dependencies
poetry add --group dev pytest black

# Instalar desde pyproject.toml
poetry install

# Actualizar dependencias
poetry update

# Ver dependencias
poetry show
poetry show --tree

# Ejecutar comando en entorno
poetry run python main.py

# Shell en entorno
poetry shell

# Build (crear wheel/sdist)
poetry build

# Publish a PyPI
poetry publish
```

¿Qué es un Paquete/Modulo en Python?

Tipo	Qué es	En el disco	Ejemplo de uso
Módulo	Un archivo <code>.py</code>	<code>math.py</code>	<code>import math</code>
Paquete	Una carpeta con <code>__init__.py</code>	<code>langchain/</code> con <code>__init__.py</code>	<code>import langchain</code>

```
/usr/lib/python3.12/
```

```
|— math.py
```

```
|— json/
```

```
|   |— __init__.py
```

```
|   |— decoder.py
```

```
|   └— encoder.py
```

```
|— os.py
```

```
|— email/
```

```
|   |— __init__.py
```

```
|   |— message.py
```

```
|   └— parser.py
```

← MÓDULO (un archivo)

← PAQUETE (carpeta)

← Archivo especial que hace esto un paquete

← MÓDULO

← PAQUETE

¿Qué es un Paquete/Modulo en Python?

¿Por qué algunos son archivos `.py` y otros carpetas?

- **Módulos simples** (como `math`, `os`, `sys`): Un solo archivo `.py` es suficiente
- **Paquetes complejos** (como `langchain`, `django`, `pandas`): Necesitan muchos archivos organizados en carpetas

Tipos de Paquetes

Built-in

`os`

`sys`

`json`

Estándar

`datetime`

`math`

Terceros (PyPI)

`langchain`

`pandas`

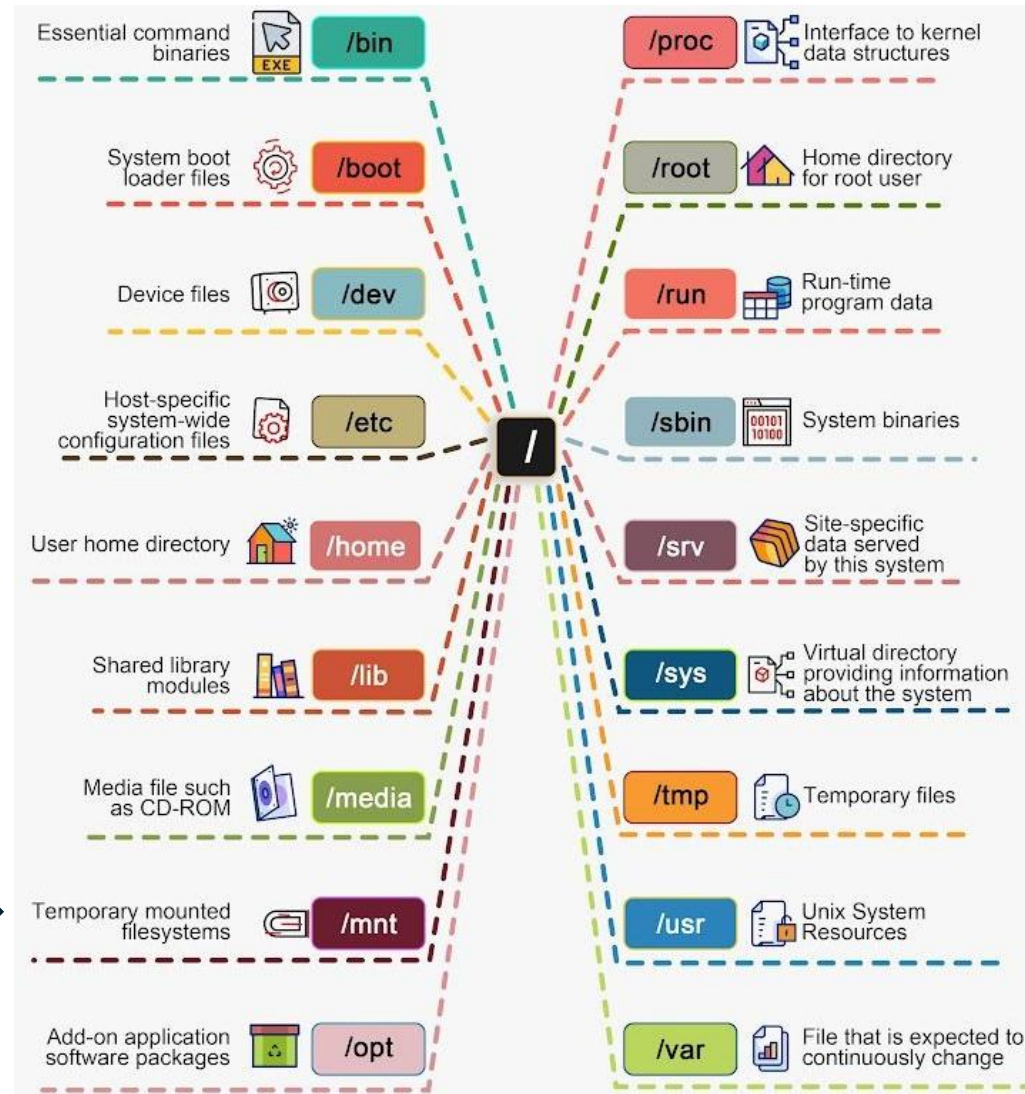
Propios

Tu código empaquetado

Anatomía de un Paquete

```
mi_paquete/  
├── __init__.py      # Lo convierte en paquete  
├── modulo1.py       # Código Python  
├── modulo2.py  
├── utils/  
│   ├── __init__.py  
│   └── helpers.py  
└── tests/  
    └── test_modulo1.py
```

¿Dónde viven los paquetes de Python?



Aquí accedes a los archivos del usuario en Linux (`cd ~`)



Aquí puedes acceder a los archivos de Windows



Aquí puedes acceder a los archivos de Windows



`/usr/bin`: aquí viven los binarios de programas, incluyendo Python.
`/usr/lib/python3.12`: aquí viven los paquetes built-in de python

¿Dónde viven los paquetes de Python?

```
xfr01@FABIANSURFACE:/usr/lib/python3.12$ ls
```

```
EXTERNALLY-MANAGED
```

```
LICENSE.txt
```

```
__future__.py
```

```
__hello__.py
```

```
__phello__
```

```
__pycache__
```

```
_aix_support.py
```

```
_collections_abc.py
```

```
_compat_pickle.py
```

```
_compression.py
```

```
_distutils_system_mod.py
```

```
_markupbase.py
```

```
_osx_support.py
```

```
_py_abc.py
```

```
_pydatetime.py
```

```
_pydecimal.py
```

```
_pyio.py
```

```
_pylong.py
```

```
_sitebuiltins.py
```

```
_strptime.py
```

```
_sysconfigdata__aarch64-linux-gnu.py
```

```
_sysconfigdata__linux_aarch64-linux-gnu.py
```

```
_threading_local.py
```

```
_weakrefset.py
```

```
abc.py
```

```
aifc.py
```

```
antigravity.py
```

```
argparse.py
```

```
ast.py
```

```
asyncio
```

```
base64.py
```

```
bdb.py
```

```
bisect.py
```

```
bz2.py
```

```
cProfile.py
```

```
calendar.py
```

```
cgi.py
```

```
cgitb.py
```

```
chunk.py
```

```
cmd.py
```

```
code.py
```

```
codecs.py
```

```
codeop.py
```

```
collections
```

```
colorsys.py
```

```
compileall.py
```

```
concurrent
```

```
config-3.12-aarch64-linux-gnu
```

```
configparser.py
```

```
contextlib.py
```

```
contextvars.py
```

```
copy.py
```

```
copyreg.py
```

```
crypt.py
```

```
csv.py
```

```
ctypes
```

```
curses
```

```
dataclasses.py
```

```
datetime.py
```

```
dbm
```

```
decimal.py
```

```
difflib.py
```

```
dis.py
```

```
doctest.py
```

```
email
```

```
encodings
```

```
ensurepip
```

```
enum.py
```

```
filecmp.py
```

```
fileinput.py
```

```
fnmatch.py
```

```
fractions.py
```

```
ftplib.py
```

```
functools.py
```

```
genericpath.py
```

```
getopt.py
```

```
getpass.py
```

```
gettext.py
```

```
glob.py
```

```
graphlib.py
```

```
gzip.py
```

```
hashlib.py
```

```
heapq.py
```

```
hmac.py
```

```
html
```

```
http
```

```
imaplib.py
```

```
imghdr.py
```

```
importlib
```

```
inspect.py
```

```
io.py
```

```
ipaddress.py
```

```
json
```

```
keyword.py
```

```
lib-dynload
```

```
linecache.py
```

```
locale.py
```

```
logging
```

```
lzma.py
```

```
mailbox.py
```

```
mailcap.py
```

```
mimetypes.py
```

```
modulefinder.py
```

```
multiprocessing
```

```
netrc.py
```

```
nntplib.py
```

```
ntpath.py
```

```
nturl2path.py
```

```
numbers.py
```

```
opcode.py
```

```
operator.py
```

```
optparse.py
```

```
os.py
```

```
pathlib.py
```

```
pdb.py
```

```
pickle.py
```

```
pickletools.py
```

```
pipes.py
```

```
pkgutil.py
```

```
platform.py
```

```
plistlib.py
```

```
poplib.py
```

```
posixpath.py
```

```
pprint.py
```

```
profile.py
```

```
pstats.py
```

```
pty.py
```

```
py_compile.py
```

```
pyclbr.py
```

```
pydoc.py
```

```
pydoc_data
```

```
queue.py
```

```
quopri.py
```

```
random.py
```

```
re
```

```
reprlib.py
```

```
rlcompleter.py
```

```
runpy.py
```

```
sched.py
```

```
secrets.py
```

```
selectors.py
```

```
shelve.py
```

```
shlex.py
```

```
shutil.py
```

```
signal.py
```

```
site.py
```

```
sitecustomize.py
```

```
smtplib.py
```

```
sndhdr.py
```

```
socket.py
```

```
socketserver.py
```

```
sqlite3
```

```
sre_compile.py
```

```
sre_constants.py
```

```
sre_parse.py
```

```
ssl.py
```

```
stat.py
```

```
statistics.py
```

```
string.py
```

```
stringprep.py
```

```
struct.py
```

```
subprocess.py
```

```
sunau.py
```

```
symtable.py
```

```
sysconfig.py
```

```
tabnanny.py
```

```
tarfile.py
```

```
telnetlib.py
```

```
tempfile.py
```

```
test
```

```
textwrap.py
```

```
this.py
```

```
threading.py
```

```
timeit.py
```

```
token.py
```

```
tokenize.py
```

```
tomllib
```

```
trace.py
```

```
traceback.py
```

```
tracemalloc.py
```

```
tty.py
```

```
turtle.py
```

```
types.py
```

```
typing.py
```

```
unittest
```

```
urllib
```

```
uu.py
```

```
uuid.py
```

```
venv
```

```
warnings.py
```

```
wave.py
```

```
weakref.py
```

```
webbrowser.py
```

```
wsgiref
```

```
xdrlib.py
```

```
xml
```

```
xmlrpc
```

```
zipapp.py
```

```
zipfile
```

```
zipimport.py
```

```
zoneinfo
```

¿Dónde viven los paquetes de Python?

<code>__future__</code>	<code>_stat</code>	<code>ftplib</code>	<code>runpy</code>	<code>_interpchannels</code>	<code>cProfile</code>	<code>msvcrt</code>	<code>timeit</code>
<code>__hello__</code>	<code>_statistics</code>	<code>functools</code>	<code>sched</code>	<code>_interpqueues</code>	<code>calendar</code>	<code>multiprocessing</code>	<code>tkinter</code>
<code>__phello__</code>	<code>_string</code>	<code>gc</code>	<code>secrets</code>	<code>_interpreters</code>	<code>cmath</code>	<code>netrc</code>	<code>token</code>
<code>_abc</code>	<code>_strptime</code>	<code>genericpath</code>	<code>select</code>	<code>_io</code>	<code>cmd</code>	<code>nt</code>	<code>tokenize</code>
<code>_aix_support</code>	<code>_struct</code>	<code>getopt</code>	<code>selectors</code>	<code>_ios_support</code>	<code>code</code>	<code>ntpath</code>	<code>tomllib</code>
<code>_android_support</code>	<code>_suggestions</code>	<code>getpass</code>	<code>shelve</code>	<code>_json</code>	<code>codecs</code>	<code>nturl2path</code>	<code>trace</code>
<code>_apple_support</code>	<code>_symtable</code>	<code>gettext</code>	<code>shlex</code>	<code>_locale</code>	<code>codeop</code>	<code>numbers</code>	<code>traceback</code>
<code>_ast</code>	<code>_sysconfig</code>	<code>glob</code>	<code>shutil</code>	<code>_lsprof</code>	<code>collections</code>	<code>opcode</code>	<code>tracemalloc</code>
<code>_asyncio</code>	<code>_thread</code>	<code>graphlib</code>	<code>signal</code>	<code>_lzma</code>	<code>colorsys</code>	<code>operator</code>	<code>tty</code>
<code>_bisect</code>	<code>_threading_local</code>	<code>gzip</code>	<code>site</code>	<code>_markupbase</code>	<code>compileall</code>	<code>optparse</code>	<code>turtle</code>
<code>_blake2</code>	<code>_tkinter</code>	<code>hashlib</code>	<code>smtplib</code>	<code>_md5</code>	<code>concurrent</code>	<code>os</code>	<code>turtledemo</code>
<code>_bz2</code>	<code>_tokenize</code>	<code>heapq</code>	<code>socket</code>	<code>_multibytecodec</code>	<code>configparser</code>	<code>pathlib</code>	<code>types</code>
<code>_codecs</code>	<code>_tracemalloc</code>	<code>hmac</code>	<code>socketserver</code>	<code>_multiprocessing</code>	<code>contextlib</code>	<code>pdb</code>	<code>typing</code>
<code>_codecs_cn</code>	<code>_typing</code>	<code>html</code>	<code>sqlite3</code>	<code>_multiprocessing</code>	<code>contextvars</code>	<code>pickle</code>	<code>unicodedata</code>
<code>_codecs_hk</code>	<code>_uuid</code>	<code>http</code>	<code>sre_compile</code>	<code>_opcode</code>	<code>copy</code>	<code>pickletools</code>	<code>unittest</code>
<code>_codecs_iso2022</code>	<code>_warnings</code>	<code>idlelib</code>	<code>sre_constants</code>	<code>_opcode_metadata</code>	<code>copyreg</code>	<code>pip</code>	<code>urllib</code>
<code>_codecs_jp</code>	<code>_weakref</code>	<code>imaplib</code>	<code>sre_parse</code>	<code>_operator</code>	<code>csv</code>	<code>pkgutil</code>	<code>uuid</code>
<code>_codecs_kr</code>	<code>_weakrefset</code>	<code>importlib</code>	<code>ssl</code>	<code>_osx_support</code>	<code>ctypes</code>	<code>platform</code>	<code>venv</code>
<code>_codecs_tw</code>	<code>_winapi</code>	<code>inspect</code>	<code>stat</code>	<code>_overlapped</code>	<code>curses</code>	<code>plistlib</code>	<code>warnings</code>
<code>_collections</code>	<code>_wmi</code>	<code>io</code>	<code>statistics</code>	<code>_pickle</code>	<code>dataclasses</code>	<code>poplib</code>	<code>wave</code>
<code>_collections_abc</code>	<code>_zoneinfo</code>	<code>ipaddress</code>	<code>string</code>	<code>_py_abc</code>	<code>datetime</code>	<code>posixpath</code>	<code>weakref</code>
<code>_colorize</code>	<code>abc</code>	<code>itertools</code>	<code>stringprep</code>	<code>_pydatetime</code>	<code>dbm</code>	<code>pprint</code>	<code>webbrowser</code>
<code>_compat_pickle</code>	<code>antigravity</code>	<code>json</code>	<code>struct</code>	<code>_pydecimal</code>	<code>decimal</code>	<code>profile</code>	<code>winreg</code>
<code>_compression</code>	<code>argparse</code>	<code>keyword</code>	<code>subprocess</code>	<code>_pyio</code>	<code>difflib</code>	<code>pstats</code>	<code>winsound</code>
<code>_contextvars</code>	<code>array</code>	<code>linecache</code>	<code>syntable</code>	<code>_pylong</code>	<code>dis</code>	<code>pty</code>	<code>wsgiref</code>
<code>_csv</code>	<code>ast</code>	<code>locale</code>	<code>sys</code>	<code>_pyrepl</code>	<code>doctest</code>	<code>py_compile</code>	<code>xml</code>
<code>_ctypes</code>	<code>asyncio</code>	<code>logging</code>	<code>sysconfig</code>	<code>_queue</code>	<code>email</code>	<code>pyclbr</code>	<code>xmlrpc</code>
<code>_datetime</code>	<code>atexit</code>	<code>lzma</code>	<code>tabnanny</code>	<code>_random</code>	<code>encodings</code>	<code>pydoc</code>	<code>xxsubtype</code>
<code>_decimal</code>	<code>base64</code>	<code>mailbox</code>	<code>tarfile</code>	<code>_sha1</code>	<code>ensurepip</code>	<code>pydoc_data</code>	<code>zipapp</code>
<code>_elementtree</code>	<code>bdb</code>	<code>marshal</code>	<code>tempfile</code>	<code>_sha2</code>	<code>enum</code>	<code>pyexpat</code>	<code>zipfile</code>
<code>_functools</code>	<code>binascii</code>	<code>math</code>	<code>textwrap</code>	<code>_sha3</code>	<code>errno</code>	<code>queue</code>	<code>zipimport</code>
<code>_hashlib</code>	<code>bisect</code>	<code>mimetypes</code>	<code>this</code>	<code>_signal</code>	<code>faulthandler</code>	<code>quopri</code>	<code>zlib</code>
<code>_heapq</code>	<code>builtins</code>	<code>mmap</code>	<code>threading</code>	<code>_sitebuiltins</code>	<code>filecmp</code>	<code>random</code>	<code>zoneinfo</code>
<code>_imp</code>	<code>bz2</code>	<code>modulefinder</code>	<code>time</code>	<code>_socket</code>	<code>fileinput</code>	<code>re</code>	
				<code>_sqlite3</code>	<code>fnmatch</code>	<code>reprlib</code>	
				<code>_sre</code>	<code>fractions</code>	<code>rlcompleter</code>	
				<code>_ssl</code>			

¿Dónde instalar otros paquetes de Python?

- **Con sudo pip install (❌ NO RECOMENDADO)**
 - /usr/lib/python3.12/dist-packages/
- **Con pip install SIN sudo (instalación de usuario ⚠️ OK pero no ideal)**
 - ~/.local/lib/python3.12/site-packages/
- **Dentro de un entorno virtual (✅ RECOMENDADO)**
 - ./venv/lib/python3.12/site-packages/

5. Laboratorios de entornos virtuales y gestión de dependencias

Entornos virtuales con venv

📁 Comparación Visual

```
/usr/lib/python3.12/
├── dist-packages/      ← Paquetes del sistema (apt install python3-*)
│   ├── apt/
│   └── setuptools/
└── *.py                ← Módulos built-in (json, os, sys, math)
```

```
/home/usuario/.local/lib/python3.12/
└── site-packages/      ← Paquetes de usuario (pip install sin venv)
    ├── langchain/
    ├── openai/
    └── requests/
```

```
/home/usuario/mi-proyecto/.venv/lib/python3.12/
└── site-packages/      ← Paquetes del venv (AISLADOS)
    ├── langchain/
    ├── openai/
    └── chromadb/
```

```
• xfrol@FABIANSURFACE:/usr/lib/python3.12$ cd venv
• xfrol@FABIANSURFACE:/usr/lib/python3.12/venv$ tree
```

```
├── __init__.py
├── __main__.py
├── pycache
│   ├── __init__.cpython-312.pyc
│   └── __main__.cpython-312.pyc
├── scripts
│   ├── common
│   │   ├── Activate.ps1
│   │   └── activate
│   └── posix
│       ├── activate.csh
│       └── activate.fish
```

5 directories, 8 files

```
• xfrol@FABIANSURFACE:/usr/lib/python3.12/venv$ cd ..
• xfrol@FABIANSURFACE:/usr/lib/python3.12$ cd re
• xfrol@FABIANSURFACE:/usr/lib/python3.12/re$ tree
```

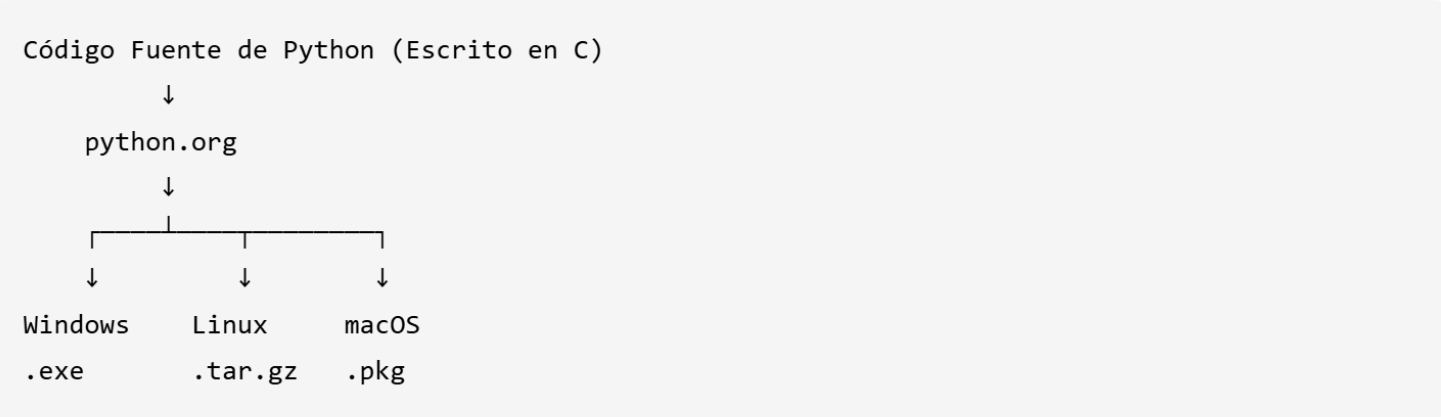
```
├── __init__.py
├── pycache
│   ├── __init__.cpython-312.pyc
│   ├── _casefix.cpython-312.pyc
│   ├── _compiler.cpython-312.pyc
│   ├── _constants.cpython-312.pyc
│   └── _parser.cpython-312.pyc
├── _casefix.py
├── _compiler.py
├── _constants.py
└── _parser.py
```

6. Métodos de ejecución de Python

Cómo funciona Python

Multiplataforma: Mismo Código Fuente, Diferentes Binarios

Python está escrito en C y se compila de manera diferente para cada sistema operativo:



SO	Descarga	Formato Binario	Ubicación de Ejemplo
Windows	python-3.11.5-amd64.exe	Ejecutable PE	C:\Python311\python.exe
Linux	python-3.11.5.tar.gz	Ejecutable ELF	/usr/bin/python3
macOS	python-3.11.5-macos.pkg	Ejecutable Mach-O	/usr/local/bin/python3

Los tres usan **CPython** (la implementación oficial de Python escrita en C), pero el binario

Cómo funciona Python

Diferentes Fuentes de Instalación - Mismo CPython

Aunque obtengas Python de diferentes fuentes, generalmente sigues obteniendo CPython:

Fuente	SO	¿Usa CPython?	Notas
python.org	Todos	 Sí	Distribución oficial
Microsoft Store	Windows	 Sí	CPython reempaquetado
apt-get (Ubuntu)	Linux	 Sí	CPython de la distribución
Homebrew (Mac)	macOS	 Sí	CPython de Homebrew
Anaconda	Todos	 Sí	CPython de Anaconda + conda
Miniconda	Todos	 Sí	Anaconda mínimo

Punto Clave: ¡Todos usan el mismo intérprete CPython, solo empaquetado de manera diferente!

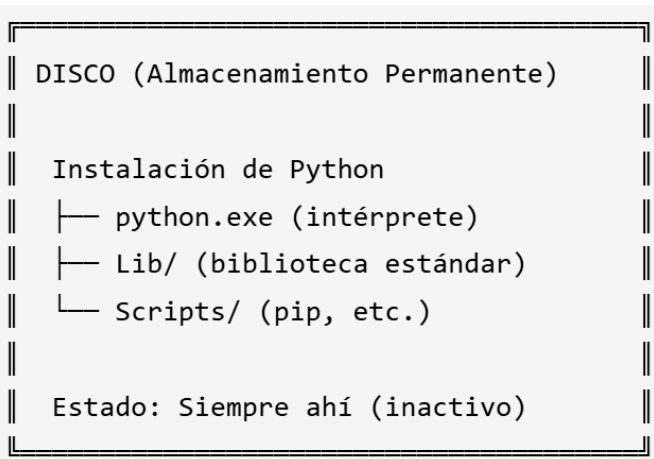
Cómo funciona Python

Comparación: Tres Métodos de Ejecución

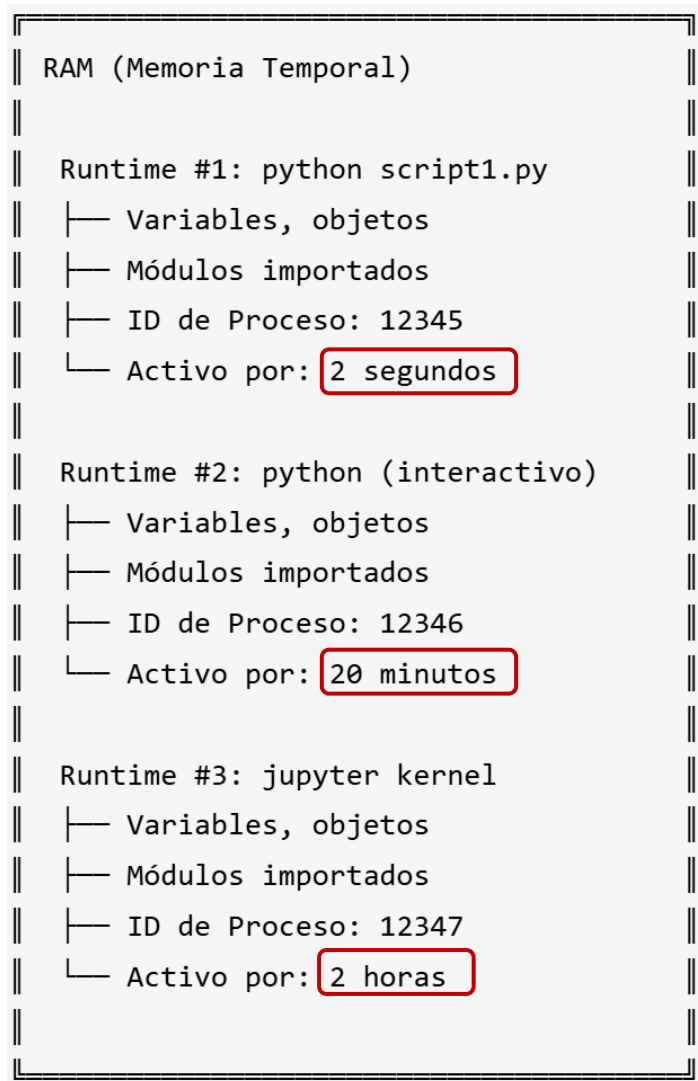
Característica	Script	REPL	Jupyter
Tipo de Archivo	.py	Ninguno	.ipynb
Duración del Runtime	Segundos	Minutos	Horas
Variables Persisten	✗ No	✓ Hasta salir	✓ ✓ Hasta reiniciar
Re-ejecutar Mismo Código	Fresco cada vez	Mismo runtime	Mismo runtime
Orden de Ejecución	De arriba a abajo	Secuencial	Cualquier orden
Visualización de Salida	Texto en terminal	Texto en terminal	Rico (gráficos, tablas)
Mejor Para	Producción	Aprendizaje	Análisis
Uso Típico	Automatización	Pruebas	Exploración

Cómo funciona Python

Qué Sucede Cuando Creas un Runtime



(Ejecutas: python)



(Los programas terminan)



[Memoria liberada]
[Runtimes destruidos]
[La instalación permanece en disco]

Cómo funciona Python

Arquitectura del Stack de Ejecución de Python

Cómo se ejecuta tu código (¡Siempre necesita Python!)



Cómo funciona Python

Arquitectura del Stack de Ejecución de Python

Cómo se ejecuta tu código (¡Siempre necesita Python!)

